



Gymnázium J. K. Tyla

DarkFactory: Agentic Engineering in practice (Agentické inženýrství v praxi)

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Rozsah práce: 6563 slov / 44700 znaků
Rozsah práce: 6563 slov / 44700 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Odborná práce zkoumá využití agentní umělé inteligence při vývoji softwaru se zaměřením na architekturu agentního harnessu. Teoretická část vymezuje mechanismy softwarového inženýrství, jazykových modelů, správy kontextu, nástrojů, řízení provádění a multiagentních systémů. Praktická část popisuje systém DarkFactory, který odděluje modelové rozhodování od trvalého stavu, prostředí, capabilities, GitHub control plane, ověřování a bezpečnostních hranic. Evaluace používá zdrojový kód, automatické testy a CI výsledky a rozlišuje implementované mechanismy od vlastností, které ještě vyžadují živý end-to-end důkaz.

Abstract

This thesis examines the use of agentic artificial intelligence in software development, focusing on the architecture of an agent harness. The theoretical part defines mechanisms from software engineering, language models, context management, tools, execution control, and multi-agent systems. The practical part describes DarkFactory, a system that separates model decisions from persistent state, the execution environment, capabilities, the GitHub control plane, verification, and security boundaries. The evaluation uses source code, automated tests, and CI results and distinguishes implemented mechanisms from properties that still require live end-to-end evidence.

Keywords (Klíčová slova)

Agent Harness (Agentní harness), Agent Loop (Smyčka ReAct) [ReAct Loop], Agentic Engineering (Agentické inženýrství), CI (Průběžná integrace) [Continuous Integration], Context Engineering (Kontextové inženýrství), Guardrail (Deterministický mantinel) [Deterministic Guardrail], HITL (Zapojení člověka do smyčky) [Human-in-the-loop], LLM (Jazykový model) [Large Language Model], Prompt Engineering (Promptové inženýrství), Sandbox (Izolované běhové prostředí), Tools (Nástroje), Version Control (Správa verzí)

Obsah

Prohlášení	2
Anotace	3
Abstract	4
Keywords (Klíčová slova)	5
1 Úvod	11
1.1 Motivace a vymezení problému	11
1.2 Cíl práce a výzkumné otázky	11
1.2.1 Hlavní cíl	11
1.2.2 Dílčí cíle	12
1.2.3 Výzkumné otázky	12
1.3 Metodika práce	12
2 Teoretická část	14
2.1 Úvod	14
2.2 Software Engineering	14
Software Engineering (Softwarové inženýrství)	14
2.2.1 AI-asistovaný vývoj	14
Vibe Coding	14
Slop	15
2.2.2 Specifikace a plánování	15
Spec-Driven Development (Vývoj řízený specifikací)	15
Planning (Plánování)	15
DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]	15
2.2.3 Řízení změn	16
Version Control (Správa verzí)	16
Branch (Větev)	16
Pull Request	16
2.2.4 Ověřování a integrace	16
CI (Průběžná integrace) [Continuous Integration]	16
Integration Test (Integrační test)	16
2.2.5 Běhová prostředí	16
Runtime (Běhové prostředí)	16
Container (Softwarový kontejner) [Software Container]	17
2.3 Model	17
2.3.1 Jazykové modely	17

Transformer	17
LLM (Jazykový model) [Large Language Model]	17
Tokenizer (Tokenizér)	17
Token	17
Embedding (Vektorová reprezentace)	18
2.3.2 Inferenční kontext	18
Context Window (Kontextové okno)	18
KV Cache (Mezipaměť klíčů a hodnot) [Key–Value Cache]	18
2.3.3 Limity modelu	19
Context Rot (Degradace kontextu)	19
Divergence (Divergence modelu) [Model Divergence]	19
2.4 Harness	19
Agent Harness (Agentní harness)	19
2.4.1 Stav běhu	21
Session (Agentní sezení) [Agent Session]	21
Turn (Tah interakce)	21
Transcript (Přepis)	21
State (Stav)	21
2.4.2 Běh a prostředí	22
Agent Loop (Smyčka ReAct) [ReAct Loop]	22
Environment (Běhové prostředí agenta) [Agent Environment]	22
Sandbox (Izolované běhové prostředí)	22
2.4.3 Rozšíření harnessu	23
Plugins (Rozšíření)	23
Tools (Nástroje)	23
JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)	23
Code Execution (Spouštění kódu)	23
Scripts (Skripty)	23
Hooks (Událostní zachytné body)	23
MCP (Model Context Protocol)	23
Skills (Dovednosti)	24
2.5 Agentic Engineering	25
Agentic Engineering (Agentické inženýrství)	25
2.5.1 Instrukce modelu	25
Prompt Engineering (Promptové inženýrství)	25
System Prompt (Systémový prompt)	25

2.5.2	Kontextové mechanismy	25
	Context Engineering (Kontextové inženýrství)	25
	Context Injection (Vkládání kontextu)	25
	Prompt Injection	26
	Compaction (Kompakce kontextu) [Context Compaction]	26
	RAG (Generování rozšířené vyhledáváním) [Retrieval-Augmented Generation]	26
2.5.3	Řízení provádění	26
	Loops (Cílené smyčky) [Goal Loops]	26
	Guardrail (Deterministický mantinel) [Deterministic Guardrail]	26
	HITL (Zapojení člověka do smyčky) [Human-in-the-loop]	26
2.5.4	Multi-Agent Systems	27
	Subagent (Podřízený agent)	27
	Orchestrator (Orchestrátor)	27
	Handoff (Předání řízení)	27
	Swarm (Roj)	27
	Graphs (Pracovní grafy) [Workflow Graphs]	27
3	Praktická část	28
3.1	Úvod	28
3.2	Návrh systému DarkFactory	28
3.2.1	Cíle návrhu	28
3.2.2	Celková architektura	28
	DarkFactory	29
3.2.3	Vykonávací jádro a stav	29
	Protocol (Protokol DarkFactory) [DarkFactory Protocol]	29
	Run State (Stav běhu)	30
	Model Routing (Směrování modelu)	30
	Supervisor (Dohled nad během) [Execution Supervisor]	30
	Result Capture (Zachycení výsledku)	30
	Recovery (Obnova běhu)	30
3.2.4	Systém capabilities	30
	Capability	31
	Capability ABI	31
	Capability Adapter (Adaptér capability)	31
3.2.5	Externí integrace	31
	GitHub Control Plane (GitHub jako řídicí vrstva)	31

3.2.6	Identita a bezpečnostní hranice	31
	Keychain (Správa strojových přihlašovacích údajů) [Machine Credential Key-chain]	32
	Auth (Autentizace uživatele) [Browser Authentication]	32
3.2.7	Rozhraní	32
	df CLI (Příkazové rozhraní df) [df Command-Line Interface]	32
	DarkFactory Web (Webové rozhraní DarkFactory)	32
	DarkFactory Docs (Kompilátor dokumentace DarkFactory) [DarkFactory Documentation Compiler]	32
3.3	Životní cyklus požadavku	33
3.3.1	Zachycení požadavku	33
	Request (Požadavek DarkFactory) [DarkFactory Request]	33
3.3.2	Plánování a schválení	33
	Planning (Plánování DarkFactory) [DarkFactory Planning]	33
	Review/Fix Loop (Smyčka revize a opravy)	33
3.3.3	Implementace	33
3.3.4	Ověření a revize	34
	Deterministic Verification (Deterministické ověření)	34
3.3.5	Finalizace a integrace	34
	Final Alignment (Finální kontrola souladu)	34
	Reconciliation (Rekonciliace stavu) [State Reconciliation]	34
3.3.6	Obnova a pokračování	34
3.4	Výsledky a diskuse	35
3.4.1	Metoda ověření	35
3.4.2	Technické výsledky	35
3.4.3	End-to-end ověření	36
3.4.4	Ověření na cílových repozitářích	36
3.4.5	Vyhodnocení cílů a výzkumných otázek	37
	Odpovědi na výzkumné otázky	37
3.4.6	Omezení	37
	Omezení evaluace	37
3.4.7	Diskuse	38
4	Závěr	39
	Seznam obrázků a tabulek	40
	Seznam příloh	41
A	Encyklopedie a rejstřík pojmů	42

Seznam zdrojů	43
---------------------	----

1 Úvod

Tato práce zkoumá, jak lze současné agentní systémy používat při vývoji softwaru tak, aby jejich autonomie byla spojena s trvalým stavem, ověřitelnými účinky a explicitními kontrolními body.

Teoretická část postupuje od principů softwarového inženýrství přes vlastnosti a limity jazykového modelu k agentnímu harnessu a technikám Agentic Engineering. Praktická část tyto mechanismy vztahuje k systému DarkFactory a odděluje popis implementované architektury od výsledků, které lze doložit testy, CI a provozními artefakty.

Práce se nezaměřuje na trénování modelů. Jazykový model je chápán jako inferenční komponenta uvnitř širšího systému, jehož spolehlivost závisí také na správě stavu, prostředí, nástrojích, verifikaci, bezpečnostních hranicích a způsobu orchestrace.

1.1 Motivace a vymezení problému

Současné agentní systémy dokážou nad softwarovým projektem provádět více kroků, pracovat se soubory a nástroji a ověřovat vlastní změny. Hlavním problémem této práce proto není samotné generování kódu, ale technická vrstva, která převádí schopnosti jazykového modelu do řízeného, obnovitelného a ověřitelného vývojového procesu.

V softwarovém inženýrství tento posun reprezentují například Codex a Claude Code, které pracují nad repozitáři, upravují soubory a používají nástroje pro build nebo testování. ^(1, 2)

Rozšíření generativní AI zároveň vytváří praktický důvod tyto systémy studovat: Microsoft AI Economy Institute pro první čtvrtletí roku 2026 odhadl používání generativní AI na 17,8 % světové populace ve věku 15–64 let. ⁽³⁾ Tento údaj v práci slouží pouze jako kontext rozšíření technologie, nikoli jako metrika kvality nebo autonomie agentních systémů.

DarkFactory slouží jako konkrétní artefakt, na kterém práce zkoumá oddělení modelu, trvalého stavu, prostředí, nástrojů a kontrolních mechanismů.

1.2 Cíl práce a výzkumné otázky

Tato část stanovuje hlavní cíl, dílčí cíle a výzkumné otázky práce. Společně vymezují rozsah návrhu DarkFactory a rámec jeho evaluace.

1.2.1 Hlavní cíl

Navrhnout a implementovat architekturu agentního harnessu pro dlouhotrvající vývoj softwaru, která odděluje jazykový model od trvalého stavu, prostředí a deterministických kontrolních mechanismů, a technicky ověřit vlastnosti této architektury na systému DarkFactory.

DarkFactory je implementační artefakt této architektury. Splnění cíle se posuzuje podle dohledatelné implementace navržených mechanismů a reprodukovatelných testovacích nebo provozních důkazů. Úplný živý Request lifecycle je samostatná úroveň ověření a nesmí být zaměněn za samotnou existenci architektury nebo úspěšné komponentové testy.

1.2.2 Dílčí cíle

- Vymezit teoretické mechanismy Software Engineering, Modelu, Harnessu a Agentic Engineering potřebné pro dlouhotrvající agentní vývoj softwaru.
- Navrhnout a implementovat DarkFactory s explicitním stavem běhu, odděleným prostředím, capability rozhraním, GitHub control plane a oddělenými hranicemi lidské a strojové identity.
- Implementovat mechanismy řízeného životního cyklu požadavku: Planning, deterministické ověření, review/fix, Final Alignment, integraci a obnovu přerušného běhu.
- Ověřit implementované mechanismy automatickými testy a CI nad konkrétním commitem.
- Ověřit přenositelnost vybraných částí řešení na konkrétních cílových repozitářích a samostatně vyhodnotit, zda existuje důkaz celého živého end-to-end Request lifecycle.
- Vztáhnout zjištěné výsledky a jejich omezení přímo k výzkumným otázkám.

Dílčí cíle oddělují teoretické vymezení, konstrukci artefaktu a jednotlivé úrovně jeho ověření. Nesplněná nebo neprokázaná úroveň evaluace proto nemusí být nahrazena silnějším tvrzením z jiné vrstvy důkazů.

1.2.3 Výzkumné otázky

- O1: Které mechanismy agentního harnessu a vývojového životního cyklu umožňují agentovi samostatně provádět softwarovou změnu, zatímco rozhodnutí s vyšším dopadem zůstávají explicitně řízena člověkem?
- O2: Jak může agentní harness omezit nebo obnovit neproduktivní či přerušovaný běh bez ztráty již ověřeného stavu a bez opakování přijatých deterministických účinků?
- O3: Jak lze oddělit trvalý stav dlouhotrvající úlohy od omezeného kontextového okna modelu tak, aby bylo možné práci po přerušení bezpečně obnovit a pokračovat v ní?

O1 sleduje řízenou autonomii, O2 odolnost provádění a O3 kontinuitu stavu přes hranice jednotlivých modelových kontextů a běhů. Otázky jsou záměrně formulovány tak, aby na ně bylo možné odpovědět konkrétními architektonickými prvky a reprodukovatelnými důkazy z implementace.

1.3 Metodika práce

Práce používá konstrukční přístup odpovídající design science: vymezuje problém a cíle řešení, navrhuje a implementuje artefakt DarkFactory a následně jej vyhodnocuje pomocí reprodukovatelných technických důkazů. ^(4,5)

Teoretická část vychází z odborných článků, standardů, protokolových specifikací a primární dokumentace současných agentních systémů. Jednotlivé mechanismy jsou rozděleny do samostatných konceptů, aby bylo možné oddělit vlastnosti **LLM (Jazykový model) [Large Language Model]**, harnessu a technik Agentic Engineering.

Předmětem práce není trénování neuronových sítí, optimalizace vah ani matematický rozbor učení modelu. **LLM (Jazykový model) [Large Language Model]** je chápán jako hotová inferenční komponenta a je popsán pouze v rozsahu potřebném pro vysvětlení architektury okolního systému.

Praktická část používá zdrojový kód, typované kontrakty, testy, workflow a generované artefakty DarkFactory jako primární důkaz skutečné implementace. Evaluace rozlišuje architektonický důkaz, automatizované funkční testy, CI nad konkrétním commitem a provozní ověření na konkrétních cílových repozitářích. Tvrzení o kompletním produkčním Request lifecycle je přijato pouze tehdy, pokud existuje reprodukovatelný živý průchod od schválení Planningu po merge a rekonsiliaci; aktuální absence tohoto důkazu je uvedena jako omezení, nikoli nahrazena architektonickým předpokladem.

Práce neprovádí statistický benchmark obecné výkonnosti agentních systémů. Odpovědi na výzkumné otázky jsou omezeny na tvrzení podporovaná konkrétními reprodukovatelnými důkazy.

2 Teoretická část

Vymezení konceptů potřebných pro pochopení současného agentního vývoje softwaru. Postupuje od Software Engineering přes Model a Harness k Agentic Engineering.

2.1 Úvod

Vymezení hranic teoretické části a vztahů mezi jejími hlavními vrstvami. Jazykový model je v práci chápán jako hotová inferenční komponenta. Teoretická část postupuje přes Software Engineering a Model k Harnessu a Agentic Engineering. Harness kolem modelu vytváří stavové, nástrojové a kontrolní mechanismy, na které navazují agentické techniky.

2.2 Software Engineering

Software Engineering (Softwarové inženýrství)

Systematické uplatňování inženýrských principů na specifikaci, vývoj, ověřování, provoz a údržbu softwarových systémů. ⁽⁶⁾

Pro tuto práci poskytuje procesní rámec, ve kterém musí být změny nejen vytvořeny, ale také řízeny, ověřeny a integrovány.

2.2.1 AI-asistovaný vývoj

Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽⁷⁾

Termín zavedl Andrej Karpathy v roce 2025 pro styl práce, ve kterém vývojář převážně přijímá a směřuje modelový výstup místo detailního čtení kódu. ^(7, 8)

Tweet Andreje Karpathyho z 2. února 2025, ve kterém popsal původní význam Vibe Coding.



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · 6.8M Views

Obrázek 1: Původní tweet Andreje Karpathyho o Vibe Coding. ^(7,9)

Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽¹⁰⁾

V této práci označuje zejména AI-generovaný software, jehož objem nebo zdánlivá úplnost převyšují jeho ověřenou funkčnost a udržitelnost.

2.2.2 Specifikace a plánování

Spec-Driven Development (Vývoj řízený specifikací)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽¹¹⁾

Specifikace odděluje požadované chování a omezení od konkrétní implementace a slouží jako společný referenční bod pro člověka i agenta. ⁽¹¹⁾

Planning (Plánování)

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn. ⁽⁶⁾

Plán rozděluje práci na kontrolovatelné části a určuje, podle jakých podmínek lze posoudit jejich dokončení. Závislosti mezi částmi práce lze reprezentovat například pomocí DAG. ⁽⁶⁾

DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]

Orientovaný graf bez orientovaných cyklů. ⁽¹²⁾

DAG lze použít jako dependency graph: uzly představují prvky nebo úlohy a orientované hrany jejich pořadí či závislosti; absence cyklu umožňuje konzistentní topologické uspořádání. ⁽¹²⁾ Konkrétní Workflow Graph nemusí být DAG, pokud podporuje návraty nebo jiné cykly.

2.2.3 Řízení změn

Version Control (Správa verzí)

Systém pro zaznamenávání historie změn souborů a práci s oddělenými liniemi vývoje. ⁽¹³⁾

Historie commitů umožňuje změny porovnávat, vracet a slučovat; větve umožňují oddělit souběžnou práci před integrací. ⁽¹³⁾

Branch (Větev)

Oddělená linie vývoje v systému správy verzí, která ukazuje na vlastní posloupnost commitů. ⁽¹³⁾

Větev umožňuje izolovat souběžnou změnu od cílové větve a později ji sloučit po kontrole nebo ověření. ^(13, 14)

Pull Request

Návrh na sloučení změn z jedné větve do jiné, kolem kterého GitHub soustřeďuje revizi, diskusi a automatické kontroly. ⁽¹⁵⁾

Pull request zpřístupňuje diff navržené změny a její stav před integrací do cílové větve. ⁽¹⁵⁾

2.2.4 Ověřování a integrace

CI (Průběžná integrace) [Continuous Integration]

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽¹⁶⁾

CI převádí část podmínek kvality do opakovatelných strojově vyhodnotitelných kontrol spouštěných nad změnami. ⁽¹⁶⁾

Integration Test (Integrační test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽⁶⁾
Integrační test zachycuje chyby vznikající ve vzájemném propojení částí systému, které izolované testování jednotlivých komponent nemusí odhalit. ⁽⁶⁾

2.2.5 Běhová prostředí

Runtime (Běhové prostředí)

V této práci označuje Runtime prostředí a systémové prostředky dostupné programu během jeho vykonávání.

Běhové prostředí vymezuje například procesy, souborový systém, síť, oprávnění a další podmínky vykonávání. Standardizované runtime specifikace mohou tyto podmínky definovat jako přenositelný kontrakt; OCI Runtime Specification například popisuje konfiguraci, běhové prostředí a životní cyklus kontejneru. ⁽¹⁷⁾

Container (Softwarový kontejner) [Software Container]

Izolované procesové prostředí vytvořené podle definované konfigurace a pravidel životního cyklu, ve kterém běží aplikace se svými potřebnými soubory a závislostmi. ⁽¹⁷⁾ Kontejnery poskytují oddělené uživatelské prostředí při sdílení jádra hostitelského operačního systému a usnadňují přenositelné zabalení aplikace a jejích závislostí. ⁽¹⁸⁾

2.3 Model

2.3.1 Jazykové modely

Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽¹⁹⁾

Při autoregresivním generování dekodér z dosavadní sekvence vytváří reprezentaci použitou k odhadu následujícího tokenu. ⁽¹⁹⁾

LLM (Jazykový model) [Large Language Model]

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽²⁰⁾

Současné LLM typicky používají architekturu **Transformer**, která při autoregresivním generování odhaduje další token z předchozí sekvence. ⁽¹⁹⁾ Práci s nástroji, stavem a prostředím zajišťuje okolní agentní runtime nebo harness, nikoli samotná textová inference. ⁽²¹⁾

Tokenizer (Tokenizér)

Komponenta, která převádí vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekodování. ⁽²²⁾

Pravidla tokenizace určují slovník a segmentaci vstupu; subword metody, například Byte Pair Encoding, umožňují skládat text z jednotek menších než celé slovo. ⁽²²⁾

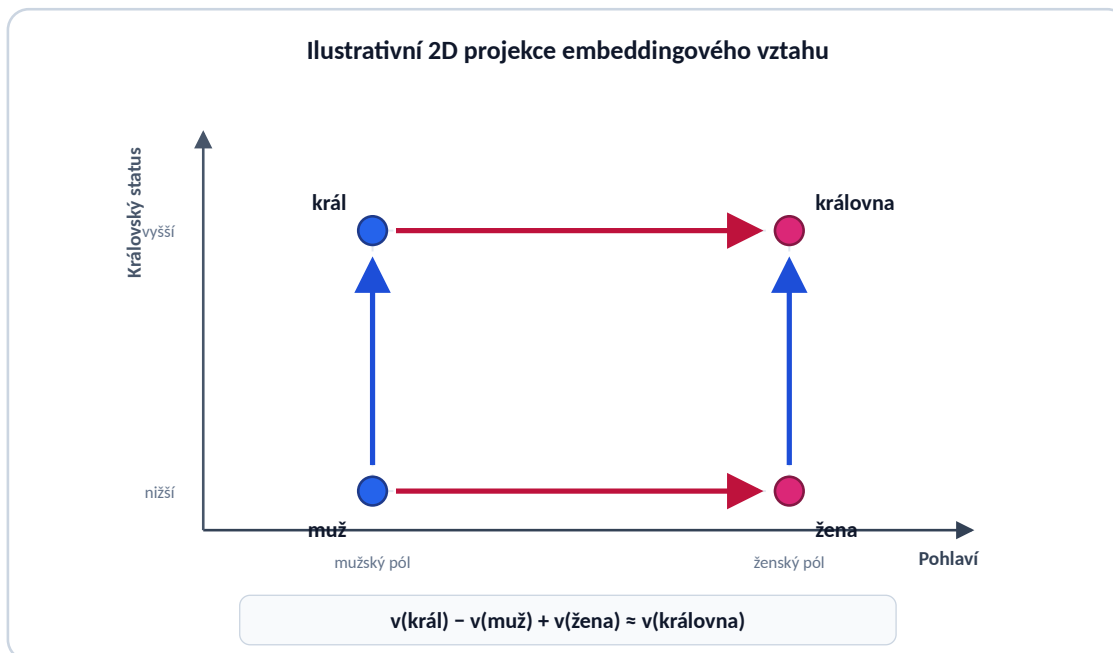
Token

Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru. ⁽²²⁾

Token nemusí odpovídat celému slovu; subword tokenizace může pracovat s částmi slov nebo jinými textovými fragmenty. ⁽²²⁾ Identifikátor tokenu je před zpracováním neuronovou sítí převeden na vektorovou reprezentaci.

Embedding (Vektorová reprezentace)

Vícerozměrná vektorová reprezentace diskrétních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽²³⁾
V jazykovém modelu embedding převádí identifikátory tokenů na spojitě vektory zpracovatelné neuronovou sítí; vektorové reprezentace lze také porovnávat podle podobnosti. ⁽²³⁾



Obrázek 2: Ilustrativní 2D projekce vztahu $\text{král} - \text{muž} + \text{žena} \approx \text{královna}$; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.

2.3.2 Inferenční kontext

Context Window (Kontextové okno)

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. ⁽²⁴⁾
Do aktivního kontextu se společně vkládají instrukce, uživatelské vstupy, historie a výsledky nástrojů. Samotná nominální délka okna nezaručuje, že model všechny vložené informace využije stejně spolehlivě. ⁽²⁴⁾

KV Cache (Mezipaměť klíčů a hodnot) [Key-Value Cache]

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. ⁽²⁵⁾
Při generování dalšího tokenu lze uložené klíče a hodnoty předchozí sekvence znovu použít místo jejich úplného přepočítání; paměťové nároky cache přitom rostou s délkou aktivní sekvence. ^(25, 26)

2.3.3 Limity modelu

Context Rot (Degradace kontextu)

V této práci označuje Context Rot pokles spolehlivosti, s níž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. ⁽²⁴⁾

Experimenty s dlouhým kontextem ukazují, že výkon může záviset na poloze relevantní informace a klesat, když je umístěna uvnitř dlouhého vstupu. ⁽²⁴⁾

Divergence (Divergence modelu) [Model Divergence]

V této práci označuje postupné odchýlení pracovní reprezentace nebo generované trajektorie modelu od zamýšleného cíle či skutečného stavu.

Divergenci mohou podporovat dlouhý nebo zahlcený kontext, ztrátová kompakce a převzetí nepřesné mezurepresentace jako dalšího vstupu. Tento pojem zde zahrnuje i jev dříve označovaný jako sémantický posun. ^(24,27)

2.4 Harness

Agent Harness (Agentní harness)

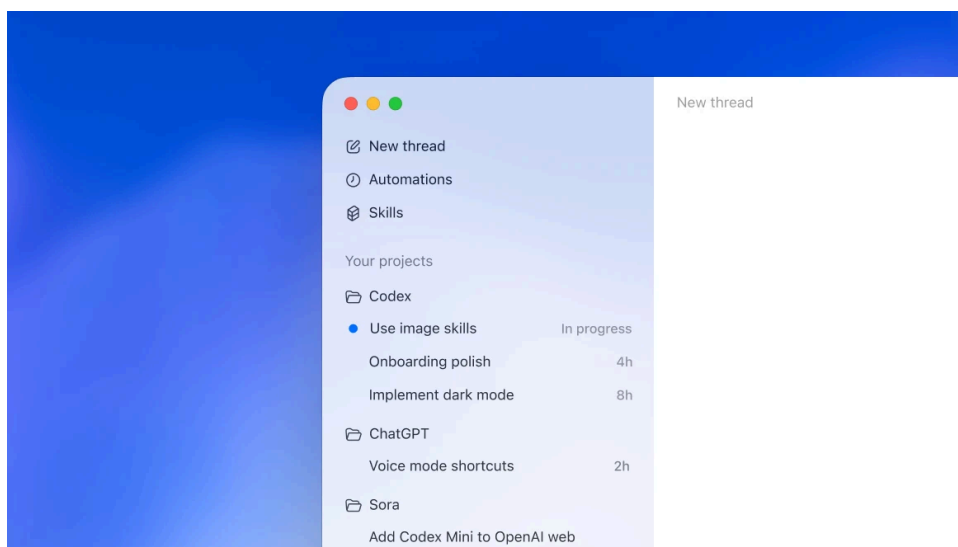
Běhová a orchestrační vrstva, která propojuje jazykový model se stavem, nástroji a prostředím. ⁽²⁸⁾

Harness řídí smyčku modelových volání a směřuje nástrojové akce do příslušné infrastruktury; může současně spravovat pracovní kontext, stav běhu a podmínky pokračování nebo ukončení. ⁽²⁸⁾ Opakované jednání v této práci reprezentuje **Agent Loop (Smyčka ReAct) [ReAct Loop]**.

Codex. Agentní vývojový systém OpenAI určený pro samostatné plnění softwarově-inženýrských úloh. ⁽¹⁾

Desktopová aplikace umožňuje řídit více agentů paralelně, oddělovat jejich práci do vláken a worktree, kontrolovat diffy a delegovat dlouhotrvající úlohy. ⁽¹⁾

Rozhraní pro práci s agentními úlohami a dovednostmi.

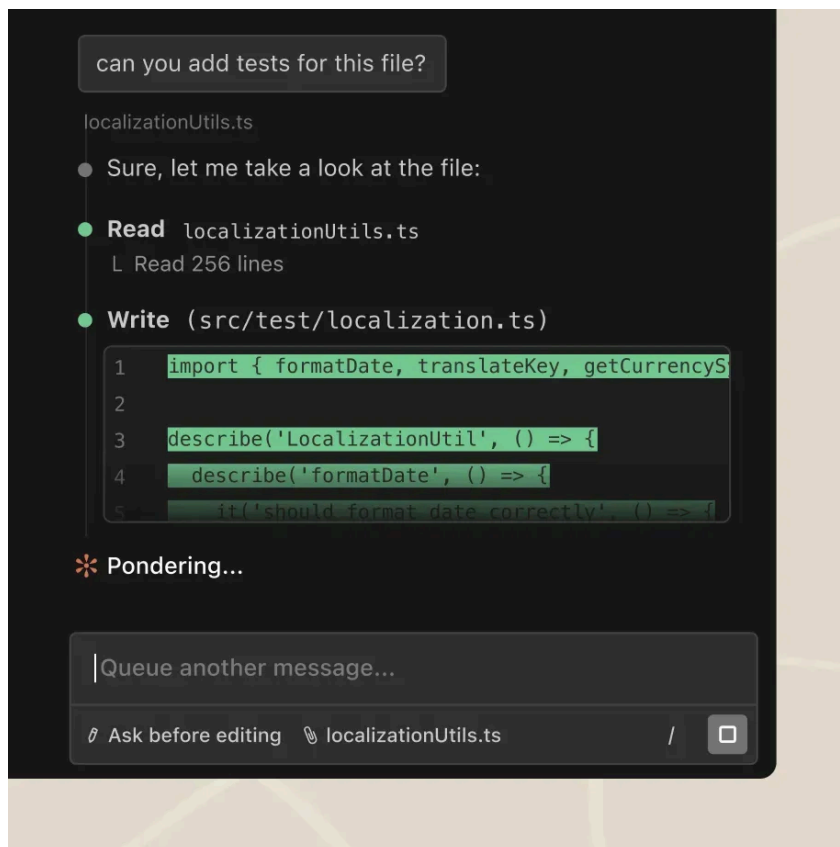


Obrázek 3: Aplikace Codex. Zdroj: OpenAI ⁽¹⁾.

Claude Code. Agentní vývojové prostředí společnosti Anthropic dostupné v terminálu, IDE, na webu a v desktopové aplikaci. ⁽²⁾

Claude Code může plánovat změny, upravovat soubory, spouštět příkazy a testy, používat Git a MCP a vytvářet pull requesty. ⁽²⁾

Čtení souboru, zápis změny a průběžný stav agentní úlohy.

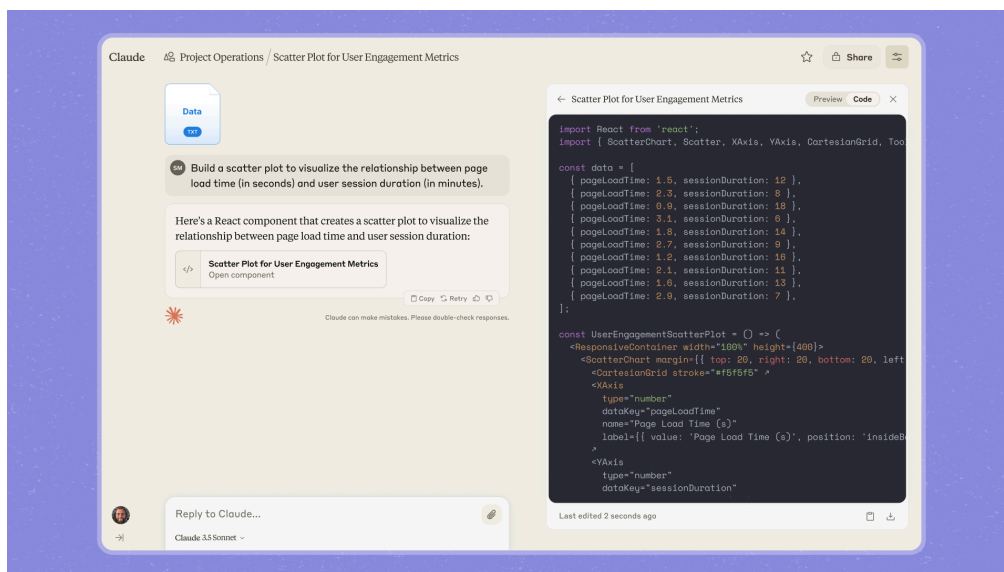


Obrázek 4: Claude Code při práci se souborem. Zdroj: Anthropic ⁽²⁾.

Claude Desktop (Aplikace Claude pro desktop) [Claude Desktop App]. Desktopová aplikační vrstva nad modely Claude, která propojuje konverzaci s lokálními soubory, aplikacemi, webem a dalšími schopnostmi. ⁽²⁹⁾

Se souhlasem uživatele může aplikace číst, upravovat a ukládat soubory a pracovat s dalšími aplikacemi. ⁽²⁹⁾

Konverzace s pracovním artefaktem a samostatným pracovním povrchem.



Obrázek 5: Aplikační rozhraní Claude. Zdroj: Anthropic ⁽²⁹⁾.

2.4.1 Stav běhu

Session (Agentní sezení) [Agent Session]

Persistovaná jednotka agentní interakce, která umožňuje navazovat na předchozí průběh mezi jednotlivými běhy. ⁽³⁰⁾

Session uchovává historii nebo záznam událostí mimo aktuální modelový kontext a při pokračování z něj lze znovu sestavit relevantní vstup. ^(28, 30) V této práci Session zastřešuje jednotlivé tahy, jejich transcript a stav potřebný pro pokračování.

Turn (Tah interakce)

Jedna diskretní jednotka interakce v konverzačním nebo agentním běhu, například vstup uživatele, výstup modelu nebo výsledek nástroje. ⁽³¹⁾

Posloupnost tahů vytváří historii session; v agentní smyčce může výsledek jednoho tahu ovlivnit rozhodnutí v následujícím tahu. ⁽³¹⁾

Transcript (Přepis)

Uspořádaný historický záznam událostí nebo položek vzniklých během Session, například zpráv, tahů a výsledků nástrojů. ⁽²⁸⁾

Transcript odpovídá na otázku, co se během běhu stalo. Není totožný se State, který zachovává aktuální pracovní skutečnosti, ani s aktivním kontextem, který obsahuje pouze informace právě předané modelu. Persistovaný transcript lze při pokračování použít k rekonstrukci potřebného kontextu. ⁽²⁸⁾

State (Stav)

Persistovaná reprezentace aktuálních skutečností a řídicích údajů, které musí harness zachovat mezi jednotlivými kroky běhu. ⁽²⁸⁾

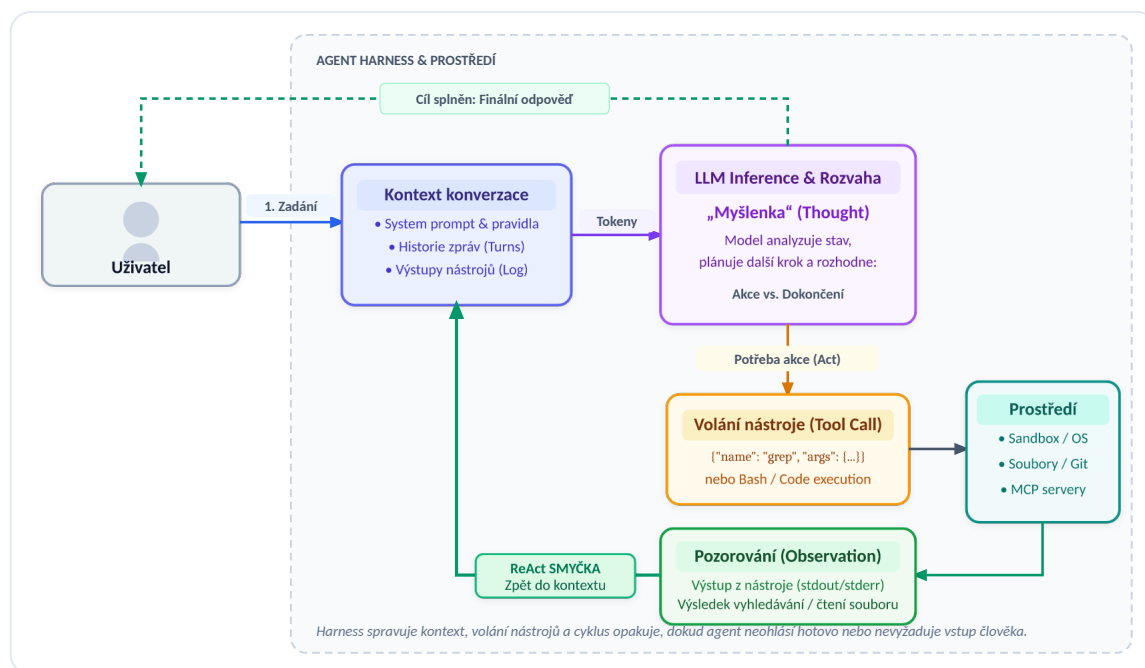
State odpovídá na otázku, co je pro pokračování právě platné. Nemusí obsahovat úplnou historii Transcriptu a nemusí být celý předán modelu; Context Engineering vybírá, která část dostupného stavu a historie vstoupí do aktivního kontextu. ⁽²⁸⁾

2.4.2 Běh a prostředí

Agent Loop (Smyčka ReAct) [ReAct Loop]

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽³¹⁾

ReAct formalizuje střídání rozhodnutí, akce a pozorování výsledku; nové pozorování se stává vstupem dalšího kroku. ⁽³¹⁾



Obrázek 6: Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu.

Environment (Běhové prostředí agenta) [Agent Environment]

Vnější prostředí, které agent prostřednictvím harnessu pozoruje a mění, například pracovní soubory, procesy, síťové služby a další systémové prostředky. ⁽²⁸⁾

Environment představuje skutečný stav světa mimo model: změna souboru nebo spuštění procesu mění prostředí, nikoli pouze textový kontext. Harness určuje, které části prostředí jsou dostupné a jakými rozhraními na ně může agent působit. ⁽²⁸⁾

Sandbox (Izolované běhové prostředí)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁸⁾ Sandbox vytváří bezpečnostní hranici kolem nedůvěryhodných účinků; konkrétní realizace může používat například kontejner nebo microVM. ^(28, 32)

2.4.3 Rozšíření harnessu

Plugins (Rozšíření)

V této práci označují Plugins distribuovatelný mechanismus pro doplnění schopností a běhového chování agentního prostředí. ⁽³³⁾

Konkrétní platformy mohou do jednoho plugin balíčku zahrnout různé druhy rozšíření, například skills, agenty, hooks nebo konfiguraci MCP serverů. ⁽³³⁾ Taxonomie této práce ale odděluje koncept Skills od Plugins a pod Plugins řadí Tools, Scripts, Hooks a MCP.

Tools (Nástroje)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽²¹⁾

Harness zprostředkuje požadavek na nástroj, provede operaci v prostředí a vrátí její výsledek modelu jako další pozorování. ^(21, 34)

JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)

Popis parametrů nástroje pomocí JSON Schema, podle kterého lze validovat strukturu vygenerovaných argumentů. ⁽³⁵⁾

Schéma určuje například povinná pole, datové typy a další omezení; structured-output mechanismy mohou generování argumentů omezit tak, aby zadanému schématu odpovídaly. ⁽³⁶⁾

Code Execution (Spouštění kódu)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽³⁷⁾

Při vývoji softwaru zpřístupňuje agentovi například testy, buildy, formátovače a diagnostické příkazy místo odhadování jejich výsledku. ⁽³⁷⁾

Scripts (Skripty)

Spustitelné soubory nebo posloupnosti příkazů používané k deterministickému provedení opakovatelné operace. ⁽³³⁾

Agentní rozšíření mohou skripty používat pro transformace, validace nebo jiné kroky, které je výhodnější provést programově než novým modelovým rozhodnutím. ^(33, 38)

Hooks (Událostní zachytné body)

Konfigurované reakce spouštěné při určených událostech životního cyklu agentního prostředí. ⁽³⁹⁾

Hook může před nebo po vybrané události spustit deterministickou logiku, například validaci, příkaz nebo jinou automatizaci. ⁽³⁹⁾

MCP (Model Context Protocol)

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽⁴⁰⁾

MCP odděluje klientskou AI aplikaci od serverů poskytujících nástroje a další schopnosti, takže integrace lze implementovat mimo vlastní jádro harnessu. ^(40, 41)

Skills (Dovednosti)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽³⁸⁾

Agent Skill je definován souborem `SKILL.md` a může odkazovat na doplňující skripty, reference nebo další zdroje načítané podle potřeby. ⁽³⁸⁾

SKILL.md (Formát SKILL.md) [SKILL.md Format]. Povinný definiční soubor Agent Skill.

Podle specifikace obsahuje YAML frontmatter následovaný instrukcemi v Markdownu;

povinnými poli frontmatteru jsou `name` a `description`. ⁽³⁸⁾

Následující úplný příklad používá povinná pole i několik volitelných polí povolených specifikací: ⁽³⁸⁾

```
---
name: repository-tests
description: Run, diagnose, and summarize the repository test suite. Use
when a software change must be validated before completion.
license: MIT
compatibility: Requires the repository's test runner to be available in
the runtime.
metadata:
  author: DarkFactory
  version: "1.0"
allowed-tools: Bash Read
---

# Repository tests

Use this skill when a change must be validated against the repository's
tests.

## Procedure

1. Detect the test command documented by the repository.
2. Run the smallest relevant test scope first.
3. Fix or report failures before continuing.
4. Run the complete required suite before completion.
5. Report the commands run and the final result.

## Failure reporting
```

For every unresolved failure, include the command, failing target, and relevant output.

Specifikace dovoluje vedle `SKILL.md` také volitelné adresáře například pro skripty, reference a assety; ty se načítají pouze podle potřeby. ⁽³⁸⁾

2.5 Agentic Engineering

Agentic Engineering (Agentické inženýrství)

V této práci zastřešuje návrh systémů, které kolem jazykového modelu zajišťují nástroje, kontext, stav, provádění a kontrolní mechanismy.

Předmětem není trénování modelu, ale inženýrství systému, který z modelových rozhodnutí vytváří řízené akce nad prostředím. ^(21, 42)

2.5.1 Instrukce modelu

Prompt Engineering (Promptové inženýrství)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁴³⁾

Prompt ovlivňuje pravděpodobnostní chování modelu, ale sám nevynucuje technickou bezpečnostní nebo autorizační hranici; pravidla, která musí systém garantovat, patří do **Guardrail (Deterministický mantinel) [Deterministic Guardrail]** nebo jiné běhové kontroly.

System Prompt (Systémový prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. ⁽⁴³⁾

Systémový prompt poskytuje stabilní instrukční kontext, ale sám o sobě není technickou izolační ani autorizační hranicí. ⁽⁴³⁾

2.5.2 Kontextové mechanismy

Context Engineering (Kontextové inženýrství)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽⁴⁴⁾

Aktivní kontext může obsahovat instrukce, popisy dostupných nástrojů, externí data a vybranou historii interakce. Jeho obsah je nutné kurátorovat vzhledem k omezené kapacitě a nerovnoměrnému využití dlouhého kontextu. ^(24, 44)

Context Injection (Vkládání kontextu)

V této práci označuje Context Injection cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok. ⁽⁴⁴⁾

Just-in-time přístup umožňuje mimo modelový kontext uchovávat odkazy nebo trvalý stav a potřebná data načíst nástrojem až během běhu. ⁽⁴⁴⁾

Prompt Injection

Manipulace chování jazykového modelu pomocí instrukcí vložených do vstupu nebo do externího obsahu, který systém následně zpracuje jako kontext. ⁽⁴⁵⁾

Přímá prompt injection přichází v uživatelském vstupu; nepřímá injection je vložena do externích dat, například webové stránky, e-mailu, dokumentu, repozitáře nebo zdroje RAG.

⁽⁴⁶⁾ Typickým příkladem je životopis obsahující skrytou instrukci, která se pokusí ovlivnit následné hodnocení kandidáta modelem, přestože dokument měl sloužit pouze jako data.

⁽⁴⁵⁾ Důsledky proto závisejí také na oprávněních a nástrojích, které má agent k dispozici. ⁽⁴⁷⁾

Compaction (Kompakce kontextu) [Context Compaction]

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽⁴⁴⁾

Kompakce uvolňuje kapacitu pro další běh, ale příliš agresivní komprese může odstranit detaily, které se později ukážou jako důležité. ^(27, 44)

RAG (Generování rozšířené vyhledávání) [Retrieval-Augmented Generation]

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁴⁸⁾

RAG odděluje znalost uloženou v externím korpusu od parametrů modelu a umožňuje vybírat podklady podle aktuálního dotazu. ⁽⁴⁸⁾

2.5.3 Řízení provádění

Loops (Cílené smyčky) [Goal Loops]

V této práci označují řídicí smyčky, které opakují jednání podle explicitního cíle, pozorovaného výsledku a podmínky dalšího pokračování nebo ukončení.

Na rozdíl od samotného běhového Agent Loopu zahrnuje Goal Loop také zpětnou vazbu k dosažení cíle, ověření výsledku a rozhodnutí, zda pokračovat, změnit postup nebo běh ukončit. ^(21, 31)

Guardrail (Deterministický mantinel) [Deterministic Guardrail]

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁴⁹⁾

Agentní frameworky mohou guardrails implementovat různými způsoby; zde je důležitá deterministicky vyhodnotitelná hranice oddělená od samotného modelového rozhodnutí.

Kontrola může běh zastavit nebo odmítnout konkrétní akci před provedením jejího účinku.

⁽⁴⁹⁾

HITL (Zapojení člověka do smyčky) [Human-in-the-loop]

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁵⁰⁾

HITL ponechává vybraná rozhodnutí člověku místo úplné automatizace. V agentním workflow může být lidský zásah explicitní přechod nebo schvalovací bod, po kterém automatizované provádění pokračuje. ⁽⁵⁰⁾

2.5.4 Multi-Agent Systems

Subagent (Podřízený agent)

Specializovaná agentní instance, které jiný agent nebo orchestrátor deleguje vymezenou dílčí úlohu. ⁽⁵¹⁾

Subagent umožňuje oddělit roli, instrukce a pracovní kontext dílčí úlohy od koordinujícího běhu a následně vrátit výsledek zpět nadřazené orchestrace. ^(51, 52)

Orchestrator (Orchestrátor)

Koordinační role nebo komponenta, která rozhoduje, kterému specializovanému agentovi předat dílčí práci a jak jeho výsledek začlenit do pokračujícího běhu. ⁽⁵¹⁾

Centralizovaný orchestrátor zůstává vlastníkem hlavního workflow a může specialisty volat jako omezené pracovní jednotky, případně jejich práci kombinovat nebo spouštět paralelně. ^(21, 51)

Handoff (Předání řízení)

Vzor koordinace, při kterém aktivní agent předá další řízení specializovanému agentovi. ⁽⁵¹⁾

Handoff se liší od centralizované orchestrace tím, že specialista není pouze zavolán jako dílčí pracovní jednotka a vrácen orchestrátoru, ale přebírá aktivní pokračování interakce nebo úlohy. ⁽⁵¹⁾

Swarm (Roj)

Decentralizovaný multiagentní vzor, ve kterém agenti lokálně předávají úlohu dalším agentům podle jejich schopností bez nutnosti jediného centrálního orchestrátoru. ⁽⁵³⁾ Jednotliví účastníci sdílejí pracovní kontext a rozhodnutí o dalším aktivním agentovi vzniká prostřednictvím handoffů mezi členy týmu. ⁽⁵³⁾

Graphs (Pracovní grafy) [Workflow Graphs]

Explicitní grafová reprezentace vícefázového workflow, v níž uzly představují dílčí práci a hrany určují přechody nebo závislosti mezi kroky. ⁽⁵²⁾ Workflow Graph může vyjádřit sekvenční, podmíněné i paralelní větvení a může koordinovat více agentů. Pokud hrany vyjadřují pouze acyklické závislosti, může mít podobu DAG; workflow s návraty nebo opakováním však obecně DAG být nemusí. ^(21, 52)

3 Praktická část

Aplikace teoretických principů na konkrétní systém DarkFactory.
Praktická část popisuje ověřenou architekturu DarkFactory, životní cyklus požadavku a doložitelné výsledky implementace.

3.1 Úvod

Vymezení způsobu, jakým praktická část aplikuje teoretické mechanismy na systém DarkFactory.

Praktická část popisuje pouze ověřenou implementaci DarkFactory a následně hodnotí chování a důkazy, které lze skutečně doložit.

3.2 Návrh systému DarkFactory

Tato část popisuje architekturu DarkFactory podle aktuální implementace a jejích veřejných balíčkových a protokolových hranic.
DarkFactory odděluje vykonávací mechanismy, verzované capabilities, GitHub integraci, credential boundaries a operátorská rozhraní do samostatných komponent s explicitními odpovědnostmi. Následující podsekcce vycházejí z aktuálního produktového kontraktu a zdrojového kódu, nikoli z plánované kompatibilní mezivrstvy nebo historické architektury.

(54)

3.2.1 Cíle návrhu

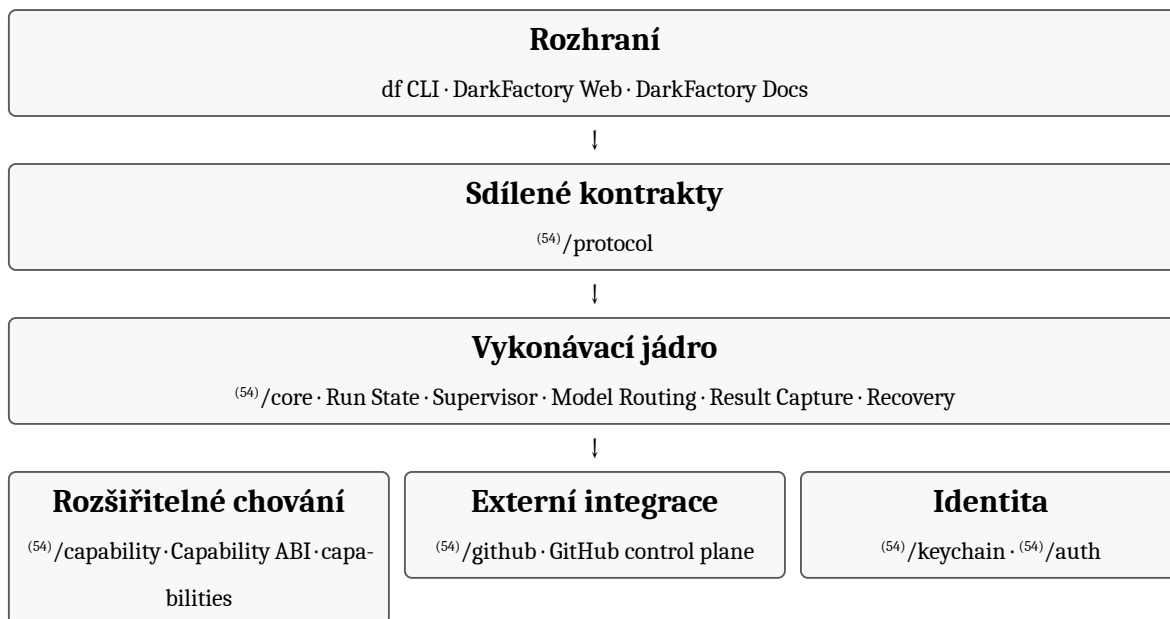
DarkFactory je navržen jako řízený autonomní systém pro vývoj softwaru, ve kterém člověk zadává záměr a schvaluje vybraná rozhodnutí, zatímco systém provádí Planning, implementaci, ověření, revizi, integraci a obnovu běhu. ⁽⁵⁴⁾

Návrh sleduje několik současně platných požadavků: běh musí být obnovitelný, skutečné změny musí být ověřovány podle pozorovaného stavu místo tvrzení modelu, agentní chování musí být rozšiřitelné mimo jádro a GitHub musí zůstat trvalou řídicí vrstvou pro Requesty, pull requesty, kontroly a autorizaci. ⁽⁵⁴⁾

3.2.2 Celková architektura

DarkFactory odděluje stabilní běhové mechanismy od agentního a doménového chování a od externích integračních hranic.

Root Bun workspace je rozdělen na protocol, core, capability, GitHub, keychain, auth, docs, CLI a web. Core vlastní provádění a stav, capability vrstva rozšiřitelné chování, GitHub trvalou vývojovou řídicí vrstvou a samostatné auth/keychain balíčky oddělují lidskou identitu od strojových přihlašovacích údajů. ⁽⁵⁴⁾



Obrázek 7: Vrstevnaté rozdělení odpovědností v architektuře DarkFactory. Schéma zobrazuje logické vlastnictví komponent, nikoli úplný graf všech importů a běhových volání.

DarkFactory

Autonomní systém pro řízený vývoj softwaru, který spojuje agentní běh, verzované capabilities a GitHub jako trvalou řídicí vrstvu. Aktuální implementace je rozdělena do samostatných balíčků pro protokol, vykonávací jádro, capabilities systém, GitHub integraci, správu přihlašovacích údajů, autentizaci, dokumentaci, CLI a webové rozhraní.

3.2.3 Vykonávací jádro a stav

Vykonávací jádro DarkFactory vlastní mechanismy, které musí zůstat konzistentní napříč jednotlivými capabilities: sdílené kontrakty, stav workflow, směrování modelů, dohled nad během, zachycení výsledků a obnovu.

`@darkfactory/protocol` poskytuje browser/runtime-safe serializované kontrakty; `@darkfactory/core` nad nimi exportuje graph/run state, routing, Supervisor, result capture a recovery. Tato hranice odděluje deterministické běhové mechanismy od agentního chování dodávaného capabilities. ⁽⁵⁴⁾

Protocol (Protokol DarkFactory) [DarkFactory Protocol]

Sdílená vrstva serializovaných schémat a kontraktů používaná mezi balíčky DarkFactory a capabilities.

Balíček `@darkfactory/protocol` exportuje kontrakty pro modely, workflow, Planning, review, kvóty, zachycení výsledků a recovery a je navržen jako bezpečný pro runtime i browserové použití.

Run State (Stav běhu)

Persistovaná reprezentace aktuálního průběhu workflow, která umožňuje vykonávacímu jádru navázat na předchozí stav místo opakování dokončených účinků. V DarkFactory patří správa graph/run state do vykonávacího jádra a používá sdílené serializované kontrakty z protokolu.

Model Routing (Směrování modelu)

Mechanismus vykonávacího jádra pro výběr vhodného modelu, poskytovatele a účtu podle požadavků úlohy, dostupnosti a provozních omezení. Model Routing je součástí `@darkfactory/core`; vyhodnocuje vhodnost kandidátů podle task kind, capability tier, kontextové kapacity, kvót a dostupnosti. Samotné přepínání mezi kandidáty při selhání řídí Supervisor.

Supervisor (Dohled nad během) [Execution Supervisor]

Komponenta vykonávacího jádra, která řídí jeden modelový běh nad uspořádanou množinou kandidátů a vynucuje jeho provozní hranice. DarkFactory Supervisor spravuje failover mezi modely a poskytovateli, kvótové a rate-limit podmínky, maximální počet tahů, časový rozpočet běhu a capability-tier escalation. Selhaný kandidát může být nahrazen dalším bez opakování již přijatých deterministických účinků.

Result Capture (Zachycení výsledku)

Mechanismus pro převod výsledku agentního kroku a pozorovaných účinků do strukturovaného, validovatelného výsledku. Aktuální implementace obsahuje schema-driven capture pro strukturované soudy i deterministické zachycení výsledku kódového kroku z reálně změněných souborů, ověřovacích akcí a commitů.

Recovery (Obnova běhu)

Mechanismus DarkFactory pro bezpečné převzetí zachovaného rozpracovaného stavu a rozhodnutí, zda lze předchozí Planning a další důkazy znovu použít. Recovery ověřuje původ zachovaného stavu, změny požadavku a base SHA, pravidla publikace a podmínky bezpečného odstranění dočasného recovery stavu.

3.2.4 Systém capabilities

Agentní a produktové chování DarkFactory není pevně zabudováno do vykonávacího jádra, ale je dodáváno verzovanými capabilities přes společné ABI. Capability může přidat nástroje, příkazy, graph-node chování, deterministické akce, verifikační pravidla, hooks, dokumentaci nebo požadavky na credentials. Z jedné kanonické TypeScript definice lze generovat podporované adaptéry, takže stejné chování nemusí mít samostatnou ručně udržovanou implementaci pro každý harness nebo integrační formát.

(54)

Capability

Verzovaný modul DarkFactory, který přidává agentní nebo produktové chování přes jednotné rozhraní.

Capability může deklarovat nástroje, příkazy, detektory, graph contributions, verifikaci, hooks, deterministické akce, credential requirements a publikační surfaces.

Capability ABI

Verzovaný kontrakt určující strukturu a kompatibilitu capability modulů DarkFactory. Aktuální ABI definuje metadata, credential requirements, nástroje, příkazy, detektory, graph contributions, verifikační pravidla, hooks, actions a další surfaces, které capability může poskytovat.

Capability Adapter (Adaptér capability)

Odvozené rozhraní, které zpřístupňuje jednu kanonickou capability jinému runtime nebo integračnímu formátu.

Současná implementace umí z capability definice vytvořit nativní in-process adaptér, Pi adaptér a MCP adaptér bez duplikace samotné capability definice.

3.2.5 Externí integrace

Externí stav vývojového procesu zůstává v systémech, které jsou jeho autoritativním vlastníkem, místo vytváření paralelní databáze uvnitř DarkFactory. GitHub v aktuální architektuře vlastní trvalý stav Requestů, pull requestů, kontrol, projektů, událostí a autorizace. DarkFactory nad ním používá typovanou integrační vrstvu `@darkfactory/github`; runtime graph může externí kontroly pozorovat, aniž by jejich stav duplikoval. ⁽⁵⁴⁾

GitHub Control Plane (GitHub jako řídicí vrstva)

Použití GitHubu jako trvalé vrstvy pro repozitáře, pull requesty, kontroly, workflow a další stav řízeného vývojového procesu DarkFactory.

DarkFactory odděluje vlastní agentní runtime od trvalého vývojového stavu a oprávnění, která zůstávají reprezentována v GitHubu a jeho integračních rozhraních.

3.2.6 Identita a bezpečnostní hranice

DarkFactory odděluje strojovou identitu a tajné credentials používané runtime od lidské identity používané webovým rozhraním.

`@darkfactory/keychain` je jediným vlastníkem strojových credentials, tokenů, GitHub App private-key operací, redakce a související diagnostiky. `@darkfactory/auth` naproti tomu poskytuje browser-safe autentizaci uživatele a session pro DarkFactory Web; browserová část nesmí importovat machine-secret implementace. ⁽⁵⁴⁾

Keychain (Správa strojových přihlašovacích údajů) [Machine Credential Keychain]

Bezpečnostní hranice DarkFactory pro držení a zpřístupňování strojových přihlašovacích údajů, tokenů a souvisejících autentizačních dat.

Balíček `@darkfactory/keychain` vlastní machine credential custody a nesmí být importován browserovými balíčky; zahrnuje mimo jiné OS keychain, vault, GitHub App credentials, OAuth, redaction a diagnostiku.

Auth (Autentizace uživatele) [Browser Authentication]

Oddělená browser-safe hranice pro lidskou autentizaci a session DarkFactory Web.

`@darkfactory/auth` odděluje uživatelskou GitHub autentizaci od machine credentials, refresh tokenů, client secrets a GitHub App private-key operací, které zůstávají mimo browserovou část.

3.2.7 Rozhraní

Stejné protokolové a stavové modely jsou zpřístupněny více operátorskými a publikačními rozhraními bez vytvoření druhého vykonávacího systému.

`@darkfactory/cli` vlastní příkaz `df` a interaktivní operátorské rozhraní, `@darkfactory/web` browserové zobrazení a operátorský web a `@darkfactory/docs` headless kompilaci dokumentačního content graphu. Vykreslení dokumentace patří webové vrstvě, nikoli druhému frontendovému runtime. ⁽⁵⁴⁾

df CLI (Příkazové rozhraní df) [df Command-Line Interface]

Veřejné příkazové a operátorské rozhraní DarkFactory pro spouštění a řízení funkcí systému.

Balíček `@darkfactory/cli` skládá veřejné příkazy nad sdílenými protocol, core a capability vrstvami.

DarkFactory Web (Webové rozhraní DarkFactory)

Browserové operátorské a renderovací rozhraní DarkFactory.

`@darkfactory/web` používá pouze browser-safe protocol, GitHub a auth kontrakty a nesmí importovat machine-secret implementace z keychain vrstvy.

DarkFactory Docs (Kompilátor dokumentace DarkFactory) [DarkFactory Documentation Compiler]

Headless dokumentační kompilátor, který vytváří typovaný obsahový graf z kanonických zdrojů projektu.

`@darkfactory/docs` vlastní konfiguraci a kompilaci dokumentace; vykreslení výsledného content graphu patří `@darkfactory/web`.

3.3 Životní cyklus požadavku

3.3.1 Zachycení požadavku

Řízený životní cyklus začíná zachycením původního zadání do trvalého Requestu před zahájením implementace.

Request zachovává verbatim znění zadání, relevantní kontext a explicitní vztahy k další práci. Tím vzniká trvalý referenční bod pro následný Planning a pro pozdější kontrolu, zda implementace stále odpovídá schválenému záměru. ⁽⁵⁴⁾

Request (Požadavek DarkFactory) [DarkFactory Request]

Trvale evidovaná jednotka práce, která zachovává původní zadání a stav doručení před zahájením implementace.

DarkFactory váže řízenou práci na Request reprezentovaný v GitHubu a používá jeho verzi, acceptance criteria a vztahy jako vstup do Planning a následného životního cyklu.

3.3.2 Plánování a schválení

Z Requestu a aktuálního stavu repozitáře vzniká jeden sjednocený Planning artefakt, který před implementací projde nezávislou revizí a jedním explicitním schválením vlastníka.

Planning obsahuje interpretaci požadavku, acceptance criteria, scope, závislosti, pořadí práce a očekávané ověření. Revize a automatická oprava se opakují do čistého výsledku; po materiální změně Requestu, base stavu, závislostí nebo recovery kontextu se předchozí schválení považuje za zastaralé. ⁽⁵⁴⁾

Planning (Plánování DarkFactory) [DarkFactory Planning]

Verzovaný a revidovaný plánovací artefakt, který převádí konkrétní Request a aktuální stav repozitáře na závazný rámec implementace a ověření.

Planning zachovává verbatim Request, acceptance criteria, behavioral contract, scope, exclusions, dependencies, sequencing, verification a známé či ještě zjišťované implementační vlastníky.

Review/Fix Loop (Smyčka revize a opravy)

Opakovaný DarkFactory mechanismus, ve kterém je Planning nebo implementace nezávisle zkontrolována, zjištění jsou strukturovaně zachycena a následná oprava pokračuje do čistého výsledku.

Runtime ukládá předmět revize, fingerprint kontextu, nalezené problémy, stav čistoty, číslo iterace a historii review/fix kroků.

3.3.3 Implementace

Po schválení Planningu runtime provádí změnu pomocí capabilities a deterministických mechanismů nad pracovním prostředím repozitáře.

Model může rozhodovat o obsahu změny a používat dostupné nástroje, ale pravdu o mutacích nevlastní jeho textový výstup. Stav souborů, diff, scope, verifikační akce, commity a další účinky jsou zjišťovány z prostředí a zachycovány vykonávacím jádrem. ⁽⁵⁴⁾

3.3.4 Ověření a revize

Výsledek implementace je nejprve deterministicky ověřen a následně prochází samotnou review/fix smyčkou. Automatické kontroly vycházejí z pozorovaných změn a skutečně spuštěných verifikačních akcí. Revize následně hledá problémy v implementaci a může vyvolat opravu a nové ověření; pokud oprava vyžaduje materiální práci mimo schválený Planning, musí být rozsah explicitně změněn a schválen. ⁽⁵⁴⁾

Deterministic Verification (Deterministické ověření)

Ověření výsledku změny pomocí přímo pozorovatelných strojových důkazů místo spoléhání na tvrzení modelu o tom, co provedl. DarkFactory odvozuje výsledek kódového kroku ze skutečně změněných souborů, scope checku, detekovaných verifikačních akcí, jejich exit stavů a případně vytvořeného commitu.

3.3.5 Finalizace a integrace

Před merge je implementace znovu porovnána se schváleným Planningem a případnými schválenými změnami rozsahu. Po Final Alignment následují požadované externí kontroly a finální review nebo merge autorizace. Teprve poté je změna integrována a DarkFactory deterministicky rekonciluje stav navázaných Requestů, pull requestů, projektu a bezpečně odstranitelných větví. ⁽⁵⁴⁾

Final Alignment (Finální kontrola souladu)

Závěrečná kontrola, která před integrací ověřuje soulad implementace s aktuálním schváleným Planningem a případnými schválenými změnami rozsahu. DarkFactory provádí Final Alignment až po deterministickém ověření a čisté implementační review/fix smyčce; teprve poté mohou následovat povinné kontroly, finální review a merge.

Reconciliation (Rekonciliace stavu) [State Reconciliation]

Sjednocení trvalého stavu Requestu, pull requestu, projektu a souvisejících workflow po dokončení nebo změně životního cyklu. DarkFactory používá kanonické stavy Backlog, ToDo, In Progress, Blocked, Done, Superseded a Dropped a po terminálních událostech aktualizuje navázaný trvalý stav podle skutečně dosaženého výsledku.

3.3.6 Obnova a pokračování

Přerušeni běhu, vyčerpání kvóty nebo konflikt nesmí samo o sobě zneplatnit již dokončené a ověřené deterministické účinky.

DarkFactory ukládá stav potřebný k pokračování a při recovery ověřuje jeho původ, změnu Requestu, base SHA a další podmínky bezpečného převzetí. Modelový nebo provider failover používá stejný princip: další kandidát pokračuje nad zachovaným stavem, místo aby bez kontroly opakoval již provedené účinky. ⁽⁵⁴⁾

3.4 Výsledky a diskuse

Tato část odděluje doložené vlastnosti implementace DarkFactory od tvrzení, která vyžadují další end-to-end nebo fleet-level důkaz. Výsledky vycházejí z konkrétního referenčního commitu, automatických testů, CI check runů a zdrojového kódu. Architektonická existence mechanismu není zaměňována za důkaz jeho úspěšného použití v celém produkčním životním cyklu.

3.4.1 Metoda ověření

Evaluace odděluje tři druhy důkazů: strukturu a kontrakty implementace, automatické testy jednotlivých mechanismů a integrační důkazy z CI nad konkrétním commitem. Jako reprodukovatelný referenční bod je použit commit `e9c10221b40589512d262a0edb95f709b923150c` z 21. září 2026. ⁽⁵⁵⁾ Nad tímto commitem byl vyhodnocen CI run `35616745304`, jeho jednotlivé joby a jejich logy. ⁽⁵⁶⁾ Architektonická tvrzení jsou kontrolována proti aktuálním balíčkovým hranicím, protokolovým kontraktům a testovanému zdrojovému kódu. Funkční tvrzení jsou přijata pouze tehdy, pokud pro ně existuje konkrétní test nebo pozorovaný workflow výsledek. Existence deklarované funkce v PRD sama o sobě není považována za důkaz úspěšného provedení.

3.4.2 Technické výsledky

Referenční CI run `35616745304` dokončil všech 15 jobů úspěšně, včetně detekce quality matrix, dokumentace, hlavní testovací sady, testů `@darkfactory/core`, `@darkfactory/keychain`, `@darkfactory/web`, capabilities a čtyř Python testovacích běhů. ⁽⁵⁶⁾

Hlavní Bun testovací sada vykazala 670 úspěšných testů v 102 souborech, 0 selhání a 2909 assertion volání. Samostatný test balíčku `@darkfactory/core` vykazal 9/9 úspěšných recovery-contract testů, `@darkfactory/keychain` 13/13 testů a vybraný webový testovací job 4/4. ⁽⁵⁶⁾

Testy současně ověřují konkrétní mechanismy používané v architektuře: atomické uložení Run State, detekci stale nebo driftovaného Planningu, zachování konverzační a nástrojové historie při provider failoveru, odvozování výsledku kódového kroku ze skutečně změněných souborů a commitů, blokování změn mimo povolený scope, recovery provenance a blokování secret-bearing recovery materiálu. ⁽⁵⁵⁾

Výsledek dokládá, že tyto mechanismy mají implementované automatické ověření a že referenční commit prošel deklarovanou quality pipeline. Neprokazuje sám o sobě úspěšnost libovolné autonomní softwarové úlohy.

3.4.3 End-to-end ověření

Testovací sada ověřuje integrované části agentního běhu a workflow, ale v referenčním evidence setu zatím není jeden reprodukovatelný živý běh, který by prošel celým produkčním životním cyklem Requestu od vytvoření zadání až po merge a následnou rekonsiliaci. Tato podmínka je zároveň explicitně vedena jako dosud nesplněná acceptance položka Requestu 359 ⁽⁵⁷⁾

Integrační testy pokrývají například přechody workflow, review/fix iterace, persistovaný Run State, události GitHubu, required-check gate, nástrojové účinky, modelový failover, result capture a recovery. ⁽⁵⁵⁾

Tato evidence dokládá spolupráci hlavních mechanismů uvnitř testovaného runtime, ale není ekvivalentní živému GitHub scénáři s reálným Requestem, schválením Planningu, vytvořením změny, pull requestem, externími kontrolami, Final Alignment, autorizovaným merge a deterministickou rekonsiliací. Dokud takový běh nebude uložen jako reprodukovatelný artefakt evaluace a acceptance v 359 zůstává otevřená, práce jej nepočítá jako splněný end-to-end důkaz. ⁽⁵⁷⁾

3.4.4 Ověření na cílových repozitářích

Fleet-level ověření rozlišuje aktivní cílové repozitáře od historických nebo archivovaných položek a hodnotí pouze konkrétní commity s dohledatelnými workflow výsledky. Repozitář `omnis` byl ověřen na commitu `a53660a1c0c6619f94768e5d520405052fb03df6`. Jeho pipeline run `34708160162` dokončil úspěšně mimo jiné web, paper a docs jobs; samostatné deploy-docs a release joby na stejném commitu byly rovněž úspěšné. ^(58, 59)

Repozitář `ChessWithQuests` byl ověřen na commitu `50a50797f29c2a636d973981993191a65df3d131`. Pipeline run `34708180783` dokončil úspěšně paper, web a docs jobs a na stejném commitu uspěly také verify-docs, deploy-docs a release kontroly. ^(60, 61)

Původně uváděné repozitáře `template-OdbornaPrace` a `OdbornaPrace-mono` jsou v době evaluace archivované a nejsou proto považovány za aktivní fleet cíle. ^(62, 63)

Aktivním repozitářem práce je `DarkFactory-Paper`, který nahrazuje starší označení `OdbornaPrace-paper`. Snapshot `5bc04974f9aed0f55295389154124a09059ff35e` prošel úspěšně CI, Deploy Documentation i Release workflow. ^(64–67)

Aktivní fleet evidence tedy v této fázi pokrývá DarkFactory, omnis, ChessWithQuests a DarkFactory-Paper. Dvě původní fleet položky jsou archivované; úplné 361 fleet acceptance proto nelze z těchto výsledků považovat za uzavřené.

3.4.5 Vyhodnocení cílů a výzkumných otázek

Hlavní cíl je v této fázi doložen na úrovni návrhu, implementovaných mechanismů a automatických testů; úplné uzavření vyžaduje ještě živý df-only Request lifecycle. Dílčí cíle zaměřené na teoretické vymezení, architekturu DarkFactory, explicitní stav, capability rozhraní, GitHub control plane, identity boundaries a automatické technické ověření mají přímé implementační nebo CI důkazy. ^(55, 56) Cíl prokázat celý řízený životní cyklus od Requestu přes schválení Planningu až po merge a následnou rekondiaci zůstává otevřený ze stejného důvodu jako end-to-end část evaluace: jednotlivé mechanismy jsou testované, ale jeden živý produkční důkaz zatím není uložen jako uzavřený artefakt.

Odpovědi na výzkumné otázky

Odpovědi na výzkumné otázky jsou omezeny na vlastnosti doložené implementací, automatickými testy, CI a dostupnými provozními artefakty.

01. Řízenou autonomii umožňuje kombinace explicitního Planning Approval, oddělené implementace, deterministického ověření, review/fix smyčky, Final Alignment a samostatné merge autorizace. Tyto mechanismy oddělují práci, kterou může systém provádět automaticky, od rozhodovacích bodů, které zůstávají pod lidskou kontrolou. Jejich dílčí kontrakty jsou implementované a testované. ^(55, 56) Bez jednoho živého průchodu celým Request lifecycle však evidence zatím neprokazuje chování této autonomie v úplném produkčním běhu; stejná podmínka zůstává otevřená v acceptance 359 ⁽⁵⁷⁾

02. Neproduktivní nebo přerušovaný běh lze omezovat pomocí turn/time budgetů, klasifikace chyb a modelového failoveru a obnovovat pomocí persistovaného Run State a recovery provenance. Testy ověřují zachování historie při failoveru, round-trip stavu a odmítnutí neplatného nebo citlivého recovery vstupu. ⁽⁵⁵⁾ Tato evidence podporuje implementovaný mechanismus obnovy; neprokazuje ještě živou crash/resume idempotenci nad reálnými externími účinky, která zůstává součástí otevřeného 359 acceptance. ⁽⁵⁷⁾

03. Trvalý stav dlouhotrvající úlohy je oddělen od context window tím, že Run State, Session a Transcript mohou existovat mimo aktuální inferenční vstup a aktivní kontext se z nich sestavuje pouze v rozsahu potřebném pro další krok. Testy ověřují persistenci Run State a recovery vazby na aktuální base a provenance. ⁽⁵⁵⁾ Tento návrh umožňuje pokračování přes hranice jednotlivých modelových běhů, ale nedokazuje nulovou informační ztrátu pro libovolně dlouhou úlohu ani univerzální kvalitu výběru nebo kompakce kontextu.

3.4.6 Omezení

Omezení evaluace

Vymezení tvrzení, která nelze z architektonického, testovacího a integračního ověření spolehlivě odvodit.

Práce nepředstavuje statistický benchmark úspěšnosti agentů, ceny, latence ani četnosti zacyklení na reprezentativním souboru úloh. Taková tvrzení by vyžadovala samostatný kontrolovaný experiment.

Aktuální evidence také ještě neobsahuje jeden živý produkční běh celého Request lifecycle ani uzavřenou fleet-level evaluaci všech cílových repozitářů. Výsledky proto zatím podporují technickou realizaci a testované vlastnosti jednotlivých mechanismů, nikoli tvrzení o obecné autonomii nebo převaze DarkFactory nad jinými agentními systémy.

3.4.7 Diskuse

Dosavadní evidence podporuje architektonické oddělení modelového rozhodování od deterministických mechanismů stavu, verifikace, credentials a obnovy, ale zatím nepodporuje obecné tvrzení o vyšší výkonnosti nebo úplné autonomii.

Silnou stránkou evaluace je přímá vazba mezi jednotlivými tvrzeními a implementačními nebo testovacími důkazy: například výsledek kódového kroku je odvozován z pracovního stromu a scope checku místo textového tvrzení modelu a recovery kontrakt je testován proti neplatné provenance i citlivému obsahu.⁽⁵⁵⁾

Slabší část evidence leží na úrovni celého systému. Jednotlivé mechanismy a jejich integrace jsou automaticky testovány a referenční quality run je zelený, ale živá end-to-end demonstrace celého Request lifecycle a ověření na cílové fleetě musí být doplněny dříve, než budou výzkumné otázky uzavřeny finálními odpověďmi.

4 Závěr

Práce ukazuje, že praktické použití agentní AI při vývoji softwaru nelze redukovat na samotné generování kódu jazykovým modelem; rozhodující část systému tvoří harness, který spravuje stav, prostředí, nástroje, ověřování a řídicí hranice.

Na systému DarkFactory byla tato architektura rozložena do explicitních runtime, capability, GitHub, identity a operátorských hranic. Referenční commit DarkFactory prošel automatickou quality pipeline a hlavní testovací sada vykazala 670 úspěšných testů bez selhání; samostatné testy ověřují mimo jiné persistenci Run State, Planning review, modelový failover, deterministické zachycení výsledku, recovery provenance a oddělení browserové autentizace od strojových credentials. ^(55,56)

Výsledky podporují architektonickou odpověď na výzkumné otázky: řízená autonomie vyžaduje explicitní schvalovací a verifikační body, odolnost běhu vyžaduje persistovaný stav a kontrolovaný failover/recovery a kontinuita dlouhotrvající úlohy nemůže být závislá pouze na modelovém context window.

Evaluace však zatím neobsahuje jeden uzavřený živý df-only průchod celým Request lifecycle od schválení Planningu po merge a rekonsiliaci; stejná podmínka zůstává otevřenou acceptance položkou Requestu 359 ⁽⁵⁷⁾ Práce proto nevyvozuje obecnou výkonnostní převahu DarkFactory ani úplnou autonomii; tato tvrzení zůstávají mimo rozsah současných důkazů.

Seznam obrázků a tabulek

Obrázek 1	Původní tweet Andreje Karpathyho o Vibe Coding. ^(7,9)	15
Obrázek 2	Ilustrativní 2D projekce vztahu král – muž + žena $\approx$ královna; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.	18
Obrázek 3	Aplikace Codex. Zdroj: OpenAI ⁽¹⁾	19
Obrázek 4	Claude Code při práci se souborem. Zdroj: Anthropic ⁽²⁾	20
Obrázek 5	Aplikační rozhraní Claude. Zdroj: Anthropic ⁽²⁹⁾	21
Obrázek 6	Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu. .	22
Obrázek 7	Vrstevnaté rozdělení odpovědností v architektuře DarkFactory. Schéma zobrazuje logické vlastnictví komponent, nikoli úplný graf všech importů a běhových volání.	29

Seznam příloh

A Encyklopedie a rejstřík pojmů	42
---------------------------------------	----

A Encyklopedie a rejstřík pojmů

Abecední přehled klíčových pojmů použitých v práci. Názvy a definice jsou odvozeny přímo z kanonických konceptů, takže rejstřík nevytváří paralelní terminologický zdroj.

Agent Harness (Agentní harness)

Běhová a orchestrační vrstva, která propojuje jazykový model se stavem, nástroji a prostředím. ⁽²⁸⁾

Agent Loop (Smyčka ReAct) [ReAct Loop]

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽³¹⁾

Agentic Engineering (Agentické inženýrství)

V této práci zastrešuje návrh systémů, které kolem jazykového modelu zajišťují nástroje, kontext, stav, provádění a kontrolní mechanismy.

CI (Průběžná integrace) [Continuous Integration]

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽¹⁶⁾

Context Engineering (Kontextové inženýrství)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽⁴⁴⁾

Guardrail (Deterministický mantinel) [Deterministic Guardrail]

V této práci označuje Guardrail programově vynucenou kontrolu, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. ⁽⁴⁹⁾

HITL (Zapojení člověka do smyčky) [Human-in-the-loop]

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. ⁽⁵⁰⁾

LLM (Jazykový model) [Large Language Model]

Velký neuronový jazykový model trénovaný na rozsáhlých textových datech pro predikci a generování posloupností tokenů. ⁽²⁰⁾

Prompt Engineering (Promptové inženýrství)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁴³⁾

Sandbox (Izolované běhové prostředí)

Oddělené běhové prostředí, ve kterém agent může spouštět kód nebo měnit pracovní soubory bez přímého přístupu ke všem prostředkům hostitelského systému. ⁽²⁸⁾

Tools (Nástroje)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému. ⁽²¹⁾

Version Control (Správa verzí)

Systém pro zaznamenávání historie změn souborů a práci s oddělenými liniemi vývoje. ⁽¹³⁾

Seznam zdrojů

1. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
2. ANTHROPIC. Claude Code. Online. 2026. [Accessed 20 září 2026]. Available from: <https://claude.com/product/claude-code>
3. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion Q1 2026: Trends and Insights. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
4. HEVNER, Alan R., MARCH, Salvatore T., PARK, Jinsoo a RAM, Sudha. Design Science in Information Systems Research. *MIS Quarterly*. 2004. Vol. 28, no. 1.
5. PEFERS, Ken, TUUNANEN, Tuure, ROTHENBERGER, Marcus A. a CHATTERJEE, Samir. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*. 2007. Vol. 24, no. 3, p. 45–77. DOI [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
6. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
7. KARPATY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
8. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
9. KRAVETS, David. A semantic history: How the term 'vibe coding' went from a tweet to prod. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.coderabbit.ai/blog/a-semantic-history-how-the-term-vibe-coding-went-from-a-tweet-to-prod>
10. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>
11. BÖCKELER, Birgitta. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. Online. 2025. [Accessed 21 září 2026]. Available from: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
12. NETWORKX DEVELOPERS. Directed Acyclic Graphs and Topological Sort. Online. 2026. [Accessed 21 září 2026]. Available from: <https://networkx.org/nx-guides/content/algorithms/dag/index.html>
13. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
14. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>

15. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
16. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
17. OPEN CONTAINER INITIATIVE. Open Container Initiative Runtime Specification. Online. 2025. [Accessed 21 září 2026]. Available from: <https://specs.opencontainers.org/runtime-spec/>
18. DOCKER. What is a container?. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
19. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
20. BROWN, Tom B., MANN, Benjamin, RYDER, Nick, SUBBIAH, Melanie, KAPLAN, Jared, DHARIWAL, Prafulla, NEELAKANTAN, Arvind, SHYAM, Pranav, SASTRY, Girish, ASKELL, Amanda, HERBERT-VOSS, Ariel, KRUEGER, Gretchen, HENIGHAN, Tom, CHILD, Rewon, RAMESH, Aditya, ZIEGLER, Daniel M., WU, Jeffrey, WINTER, Clemens, HESSE, Christopher, CHEN, Mark, SIGLER, Eric, LITWIN, Mateusz, GRAY, Scott, CHESSE, Benjamin, CLARK, Jack, BERNER, Christopher, MCCANDLISH, Sam, RADFORD, Alec, SUTSKEVER, Ilya a AMODEI, Dario. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33. Available from: <https://arxiv.org/abs/2005.14165>
21. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
22. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
23. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
24. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long

- Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
25. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
 26. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
 27. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
 28. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>
 29. ANTHROPIC. Navigating the Claude desktop app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://academy.claude.com/tutorials/navigating-the-claude-desktop-app>
 30. OPENAI. Sessions | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sessions/>
 31. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
 32. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
 33. ANTHROPIC. Create plugins | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/plugins>
 34. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>

35. WRIGHT, Austin, ANDREWS, Henry, HUTTON, Ben a DENNIS, Greg. JSON Schema Draft 2020-12. Online. 2022. [Accessed 20 září 2026]. Available from: <https://json-schema.org/draft/2020-12>
36. OPENAI. Introducing Structured Outputs in the API. Online. 2024. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>
37. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>
38. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
39. ANTHROPIC. Hooks reference | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/hooks>
40. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
41. ANTHROPIC. Connect Claude Code to tools via MCP. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/mcp>
42. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
43. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
44. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
45. OWASP GENAI SECURITY PROJECT. LLM01:2025 Prompt Injection. Online. 2025. [Accessed 21 září 2026]. Available from: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
46. OWASP FOUNDATION. LLM Prompt Injection Prevention Cheat Sheet. Online. 2026. [Accessed 21 září 2026]. Available from: https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
47. OPENAI. Understanding prompt injections. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.com/safety/prompt-injections/>
48. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄ-

- SCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
49. OPENAI. Guardrails | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/guardrails/>
 50. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
 51. OPENAI. Agent orchestration | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: https://openai.github.io/openai-agents-python/multi_agent/
 52. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
 53. MICROSOFT. Swarm | AutoGen. Online. 2026. [Accessed 21 září 2026]. Available from: <https://microsoft.github.io/autogen/dev/user-guide/agentchat-user-guide/swarm.html>
 54. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
 55. MARIUS, Patrik. DarkFactory commit e9c10221. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>
 56. GITHUB ACTIONS. DarkFactory CI run 35616745304. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/actions/runs/35616745304>
 57. MARIUS, Patrik. Request #359: finish the df production engine and retire Python orchestration. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/issues/359>
 58. MARIUS, Patrik. omnis commit a53660a1. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/commit/a53660a1c0c6619f94768e5d520405052fb03df6>
 59. GITHUB ACTIONS. omnis pipeline run 34708160162. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/omnis/actions/runs/34708160162>

60. MARIUS, Patrik. ChessWithQuests commit 50a50797. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/commit/50a50797f29c2a636d973981993191a65df3d131>
61. GITHUB ACTIONS. ChessWithQuests pipeline run 34708180783. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/ChessWithQuests/actions/runs/34708180783>
62. MARIUS, Patrik. template-OdbornaPrace repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/template-OdbornaPrace>
63. MARIUS, Patrik. OdbornaPrace-mono repository. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/OdbornaPrace-mono>
64. MARIUS, Patrik. DarkFactory-Paper commit 5bc04974. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/commit/5bc04974f9aed0f55295389154124a09059ff35e>
65. GITHUB ACTIONS. DarkFactory-Paper CI run 35617820423. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820423>
66. GITHUB ACTIONS. DarkFactory-Paper Deploy Documentation run 35617820271. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820271>
67. GITHUB ACTIONS. DarkFactory-Paper Release run 35617820286. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory-Paper/actions/runs/35617820286>
68. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>